LeetCode HOT 100

PART 10

Backtracking

回溯

55. 全排列 (No.46) MEDIUM

给定一个不含重复数字的数组 `nums` ，返回其 *所有可能的全排列*。你可以 **按任意顺序** 返回答案。

示例 1:

> **输入:** `nums = [1,2,3]`
> **输出:** `[[1,2,3],[1,3,2],[2,1,3],[2,3,1],[3,1,2],[3,2,1]]`

示例 2:

> **输入:** `nums = [0,1]`
> **输出:** `[[0,1],[1,0]]`

示例 3:

> **输入:** `nums = [1]`
> **输出:** `[[1]]`

提示:

- `1 <= nums.length <= 6`
- `-10 <= nums[i] <= 10`
- `nums` 中的所有整数 **互不相同**

55. 全排列 (No.46) MEDIUM

Solution 1 – 回溯

TIPS：回溯 → ①边界②修改③递归④复原

思路

回溯专题中个人喜欢用pos表示当前位置，ans表示当前分支的结果(本题原地修改没有ans)。递归到pos时，pos之前的数都已确定，从pos向后逐个枚举选择谁做接下去第一个数，选中的数和pos交换位置继续递归，回溯时换回来复原。

注意边界需要返回数组的拷贝，否则添加指针后res会随着nums的修改而修改！

时间复杂度：$\mathcal{O}(n \cdot n!)$，因为边界复制数组耗费$\mathcal{O}(n)$；

空间复杂度：$\mathcal{O}(n)$

```python
class Solution:
    def permute(self, nums: List[int]) -> List[List[int]]:
        def backtrack(pos):
            # 正在选当前分支的第pos个数，pos之前的数都不动了
            if pos == n:  # 最后一个数填完把当前nums返回
                res.append(nums[:])  # 易错：拷贝而不是传指针
                return
            # 每个位置都和pos互换并递归，回溯后换回来复原
            for i in range(pos, n):
                nums[pos], nums[i] = nums[i], nums[pos]
                backtrack(pos+1)
                nums[pos], nums[i] = nums[i], nums[pos]

        n, res = len(nums), []
        backtrack(0)
        return res
```

55. 全排列 (No.46) MEDIUM

Solution 2 – 递归

思路

这题也可以直接递归，也是相同的思路，每个数轮流打头，其他剩下的数继续递归。由于直接递归是手动传入临时数组的，因此回溯只有销毁临时变量的动作，而这个动作由程序自动完成，不需要其他的手动复原操作。

```python
class Solution:
    def permute(self, nums: List[int]) -> List[List[int]]:
        if len(nums) == 1:  # nums仅1个元素直接返回
            return [nums]
        res = []
        for num in nums:
            # 选num做第一个数，所有num以外的数继续全排列，得到子全排列
            sub_permute = self.permute([x for x in nums if x != num])
            # num拼上子全排列返回
            for sub in sub_permute:
                res.append([num] + sub)
        return res
```

时间复杂度：$\mathcal{O}(n \cdot n!)$，因为递归传参时构造临时数组耗费$\mathcal{O}(n)$；

空间复杂度：$\mathcal{O}(n^2)$，仍然是因为临时数组耗费$\mathcal{O}(n)$

56. 子集 (No.78) MEDIUM

给你一个整数数组 `nums` ，数组中的元素 **互不相同** 。返回该数组所有可能的子集（幂集）。

解集 **不能** 包含重复的子集。你可以按 **任意顺序** 返回解集。

示例 1：

> **输入：** `nums = [1,2,3]`
> **输出：** `[[],[1],[2],[1,2],[3],[1,3],[2,3],[1,2,3]]`

示例 2：

> **输入：** `nums = [0]`
> **输出：** `[[],[0]]`

提示：

- `1 <= nums.length <= 10`
- `-10 <= nums[i] <= 10`
- `nums` 中的所有元素 **互不相同**

56. 子集 (No.78) MEDIUM

Solution 1 – 回溯

TIPS：回溯修改操作两种思路 ①枚举选哪个 ②选或不选

思路

经典的“选或不选”回溯模板。

构造子集时，每个元素要么选要么不选。

①对于选的情况：

边界：pos到数组末尾

修改：该分支选上nums[pos]

递归：继续到pos+1

复原：把nums[pos]取出

②对于不选的情况：

无需修改和复原，边界和递归同上。

时间复杂度：$\mathcal{O}(n \cdot 2^n)$

空间复杂度：$\mathcal{O}(n)$

```
class Solution:
    def subsets(self, nums: List[int]) -> List[List[int]]:
        def backtrack(ans, pos):
            if pos == n:
                # 易错：添加数组拷贝而非指针
                res.append(ans[:])
                return
            # 不选nums[pos]
            backtrack(ans, pos+1)
            # 选nums[pos]
            ans.append(nums[pos])
            backtrack(ans, pos+1)
            ans.pop()

        res, n = [], len(nums)
        backtrack([], 0)
        return res
```

56. 子集 (No.78) MEDIUM

Solution 2 – 二进制枚举

思路

二进制枚举就是专门解决子集枚举问题的。子集枚举一共有2^n(1<<n)种情况，mask从1到2^n遍历时，其二进制位上的每一位，=1表示选，=0表示不选，n位形成了n个元素的一种可能组合。因此用i逐位与mask进行位与运算，表示第i个元素选或不选。一轮循环结束后完成一个分支，把当前分支的选集ans加入结果res中。

```
class Solution:
    def subsets(self, nums: List[int]) -> List[List[int]]:
        res, ans = [], []  # ans表示临时分支的结果
        n, mask = len(nums), 0  # mask是掩码，决定每个元素选还是不选
        while mask < 1 << n:
            ans = []
            for i in range(n):
                if mask & (1 << i):  # 当前位i选中
                    ans.append(nums[i])
            res.append(ans)
            mask += 1
        return res
```

时间复杂度：$\mathcal{O}(n \cdot 2^n)$，**空间复杂度**：$\mathcal{O}(n)$

57. 电话号码的字母组合 (No.17) MEDIUM

给定一个仅包含数字 `2-9` 的字符串，返回所有它能表示的字母组合。答案可以按 **任意顺序** 返回。

给出数字到字母的映射如下（与电话按键相同）。注意 1 不对应任何字母。

示例 1：

> **输入：** `digits = "23"`
> **输出：** `["ad","ae","af","bd","be","bf","cd","ce","cf"]`

示例 2：

> **输入：** `digits = "2"`
> **输出：** `["a","b","c"]`

提示：

- `1 <= digits.length <= 4`
- `digits[i]` 是范围 `['2', '9']` 的一个数字。

57. 电话号码的字母组合 (No.17) MEDIUM

思路

经典的“枚举所有选项”回溯模板。

对于第pos个数字digits[pos]，其对应的每个字母都可以选，因此遍历num_map中其映射的所有字母。

回溯模板和[56.子串]几乎完全一致，无非之前只有选和不选两种情况，这里有多种字母可以选择，因此用for循环实现枚举。

时间复杂度：$\mathcal{O}(3^m \times 4^n)$，其中$m$和$n$分别为digits中映射3个和4个字母的数字数量（即n为7,9的数量，m是其他数字数量）

空间复杂度：$\mathcal{O}(m+n)$

```python
class Solution:
    def letterCombinations(self, digits: str) -> List[str]:
        def backtrack(ans, pos):
            if pos == n:
                res.append("".join(ans))  # 注意返回字符串
                return
            for char in num_map[digits[pos]]:
                ans.append(char)  # 选择当前char
                backtrack(ans, pos+1)
                ans.pop()  # 去除char复原

        num_map={
            '2': "abc",
            '3': "def",
            '4': "ghi",
            '5': "jkl",
            '6': "mno",
            '7': "pqrs",
            '8': "tuv",
            '9': "wxyz"
        }
        res, n = [], len(digits)
        backtrack([], 0)
        return res
```

58. 组合总和 (No.39) MEDIUM

给你一个 **无重复元素** 的整数数组 `candidates` 和一个目标整数 `target` ，找出 `candidates` 中可以使数字和为目标数 `target` 的 所有 **不同组合** ，并以列表形式返回。你可以按 **任意顺序** 返回这些组合。

`candidates` 中的 **同一个** 数字可以 **无限制重复被选取** 。如果至少一个数字的被选数量不同，则两种组合是不同的。

对于给定的输入，保证和为 `target` 的不同组合数少于 `150` 个。

示例 1：

> **输入：** `candidates = [2,3,6,7], target = 7`
> **输出：** `[[2,2,3],[7]]`
> **解释：**
> 2 和 3 可以形成一组候选，2 + 2 + 3 = 7 。注意 2 可以使用多次。
> 7 也是一个候选， 7 = 7 。
> 仅有这两种组合。

示例 2：

> **输入：** `candidates = [2,3,5], target = 8`
> **输出：** `[[2,2,2,2],[2,3,3],[3,5]]`

示例 3：

> **输入：** `candidates = [2], target = 1`
> **输出：** `[]`

提示：

- `1 <= candidates.length <= 30`
- `2 <= candidates[i] <= 40`
- `candidates` 的所有元素 **互不相同**
- `1 <= target <= 40`

58. 组合总和 (No.39) MEDIUM

思路

当前分支答案ans，位置pos，target是距离扣减为0还需要的数值。对于pos处的数值，要么选要么不选：如果选，由于允许重复，下一轮还继续从pos开始继续选；如果不选，此分支后就再也不选，从而保证ans之间彼此不重复。这种强行按顺序选择的思路可以解决很多重复的问题。

```
class Solution:
    def combinationSum(self, candidates: List[int], target: int) -> List[List[int]]:
        def backtrack(ans, pos, target):
            # 当前分支答案ans，位置pos，距离目标差距target
            if pos == n or target <= 0:  # candidate都是正数，一旦为负就失败
                if not target:  # 刚好总和为target
                    # 易错：返回拷贝而非指针
                    res.append(ans[:])
                return
            # 每个元素选或不选
            backtrack(ans, pos+1, target)  # 一旦不选，此后再也不选
            ans.append(candidates[pos])
            backtrack(ans, pos, target - candidates[pos])  # 如果选，可能还再选
            ans.pop()  # 复原

        res, n = [], len(candidates)
        backtrack([], 0, target)
        return res
```

本题时间复杂度和空间复杂度依赖于可行解的个数，上界分别是$\mathcal{O}(n \cdot 2^n)$和$\mathcal{O}(\text{target})$。

59. 括号生成 (No.22) MEDIUM

数字 `n` 代表生成括号的对数，请你设计一个函数，用于能够生成所有可能的并且 **有效的** 括号组合。

示例 1：

> **输入：** `n = 3`
> **输出：** `["((()))","(()())","(())()","()(())","()()()"]`

示例 2：

> **输入：** `n = 1`
> **输出：** `["()"]`

提示：

- `1 <= n <= 8`

59. 括号生成 (No.22) MEDIUM

思路

ans仍为当前分支的结果，left和right是左右括号的余额，生成中左括号余额必须不大于右括号，否则不合法直接返回。如果余额均为0，说明已经生成完了；如果左括号还有余额就尝试递归一个左括号，否则就单独一个右括号。

时间复杂度：$\mathcal{O}(C(2n,n))$，其中C表示组合数，利用Stirling公式可以近似为$\mathcal{O}\left(\frac{4^n}{\sqrt{n}}\right)$；**空间复杂度**：$\mathcal{O}(n)$

```python
class Solution:
    def generateParenthesis(self, n: int) -> List[str]:
        def backtrack(ans, left, right):
            # 当前分支答案ans，还有left和right个左右括号需要生成
            if left == 0 and right == 0:  # 匹配完成
                res.append("".join(ans))
            elif right >= left:  # 右括号必须比左括号余额多
                if left > 0:  # 如果还可以生成左括号
                    ans.append('(')
                    backtrack(ans, left-1, right)
                    ans.pop()  # 复原
                # 只有右括号可以生成了
                ans.append(')')
                backtrack(ans, left, right-1)
                ans.pop()  # 复原

        res = []
        backtrack([], n, n)
        return res
```

回溯时间复杂度=递归树叶子节点树*树高，本题叶子节点由卡特兰数公式共$\frac{C(2n,n)}{n+1}$个，递归深度为n。

60. 单词搜索 (No.79) MEDIUM

给定一个 `m x n` 二维字符网格 `board` 和一个字符串单词 `word` 。如果 `word` 存在于网格中，返回 `true` ；否则，返回 `false` 。

单词必须按照字母顺序，通过相邻的单元格内的字母构成，其中“相邻”单元格是那些水平相邻或垂直相邻的单元格。同一个单元格内的字母不允许被重复使用。

示例 1：

A	B	C	E
S	F	C	S
A	D	E	E

> **输入：** board = [['A','B','C','E'],['S','F','C','S'],['A','D','E','E']], word = "ABCCED"
> **输出：** true

示例 2：

A	B	C	E
S	F	C	S
A	D	E	E

> **输入：** board = [['A','B','C','E'],['S','F','C','S'],['A','D','E','E']], word = "SEE"
> **输出：** true

示例 3：

A	B	C	E
S	F	C	S
A	D	E	E

> **输入：** board = [['A','B','C','E'],['S','F','C','S'],['A','D','E','E']], word = "ABCB"
> **输出：** false

提示：

- `m == board.length`
- `n = board[i].length`
- `1 <= m, n <= 6`
- `1 <= word.length <= 15`
- `board` 和 `word` 仅由大小写英文字母组成

进阶： 你可以使用搜索剪枝的技术来优化解决方案，使其在 `board` 更大的情况下可以更快解决问题？

60. 单词搜索 (No.79) MEDIUM

思路

边界:pos到尾返回True或者下标越界返回False；修改&递归:如果board[i][j]没被访问过并和word[pos]对上，继续向四周递归；复原：如果四周递归全部失败，返回False之前要把访问标记去除。主函数里只需要遍历每个位置作为word的出发点开始递归即可。

```python
class Solution:
    def exist(self, board: List[List[str]], word: str) -> bool:
        def backtrack(i, j, pos):
            # 当前在board[i][j]要搜索word[pos]，如果不可行返回False
            if pos == l:
                return True
            if i < 0 or i >= m or j < 0 or j >= n:
                return False
            if board[i][j] == word[pos] and (i, j) not in visited:
                visited.add((i, j))  # 记录访问
                if backtrack(i-1, j, pos+1) or backtrack(i+1, j, pos+1) or backtrack(i, j-1, pos+1) or backtrack(i, j+1, pos+1):
                    return True
                else:
                    visited.remove((i, j))  # 没有找到一条可行的路径，要复原
                    return False
            return False

        l, m, n = len(word), len(board), len(board[0])
        visited = set()  # 访问过的位置
        for i in range(m):
            for j in range(n):
                if backtrack(i, j, 0):
                    return True
        return False
```

时间复杂度上界：$\mathcal{O}(mn \cdot 3^l)$，其中$m,n$为board长宽，$l$为word长度，底数为3是因为除了出发点，其余点其实最多向三个方向递归（毕竟肯定上一轮从某个方向来的）；**空间复杂度**：$\mathcal{O}(mn)$，主要是visited最多存储$\mathcal{O}(mn)$个位置

61. 分割回文串 (No.131) MEDIUM

给你一个字符串 `s`，请你将 `s` 分割成一些 子串，使每个子串都是 **回文串** 。返回 `s` 所有可能的分割方案。

示例 1：

> **输入：** `s = "aab"`
> **输出：** `[["a","a","b"],["aa","b"]]`

示例 2：

> **输入：** `s = "a"`
> **输出：** `[["a"]]`

提示：

- `1 <= s.length <= 16`
- `s` 仅由小写英文字母组成

61. 分割回文串 (No.131) MEDIUM

Solution 1.1 – 回溯(选或不选)

思路

可以用经典的“选或不选”回溯模板，这里的修改是对当前位置选择切割或者不切割。注意必须满足回文串才能切割，或者到了最后一个字符位置必须切割。这题要多用一个string来记录当前分支ans下当前的回文子串。

时间复杂度：$\mathcal{O}(n \cdot 2^n)$，状态叶子节点$\mathcal{O}(2^n)$，深度$\mathcal{O}(n)$；

空间复杂度：$\mathcal{O}(n^2)$，递归栈调用$\mathcal{O}(n)$，每轮递归中临时数组string耗费$\mathcal{O}(n)$

```python
class Solution:
    def partition(self, s: str) -> List[List[str]]:
        def backtrack(string, ans, pos):
            # 当前回文子串string，当前分支答案ans，当前位置pos
            if pos == n:
                res.append(ans[:])  # 易错：返回拷贝而非指针
                return
            string.append(s[pos])
            # 不切割，但注意pos=n-1时必须切割
            if pos < n - 1:
                backtrack(string, ans, pos+1)
            # 切割，必须满足回文
            if string == string[::-1]:  # 回文子串
                ans.append("".join(string))
                backtrack([], ans, pos+1)
                ans.pop()  # 复原ans
            string.pop()  # 复原string

        res, n = [], len(s)
        backtrack([], [], 0)
        return res
```

61. 分割回文串 (No.131) MEDIUM

Solution 1.2 – 回溯(枚举所有选项)

思路

这题也可以用“枚举所有选项”的经典回溯模板，枚举所有下一个切割点的位置，这样就不需要用string来实时记录当前回文子串，因为一旦选好切割点，回文子串立刻生成。

时间复杂度：$\mathcal{O}(n \cdot 2^n)$

空间复杂度：$\mathcal{O}(n^2)$

```python
class Solution:
    def partition(self, s: str) -> List[List[str]]:
        def backtrack(ans, pos):
            # 当前回文子串string，当前分支答案ans，当前位置pos
            if pos == n:
                res.append(ans[:])  # 易错：返回拷贝而非指针
                return
            for end in range(pos, n):  # 枚举所有下一个切割点
                # 字符串逆置不能用s[end:pos-1:-1]，pos-1等于-1会被当作最后一个元素
                if s[pos:end+1] == s[pos:end+1][::-1]:  # 回文
                    ans.append(s[pos:end+1])
                    backtrack(ans, end+1)
                    ans.pop()  # 复原ans

        res, n = [], len(s)
        backtrack([], 0)
        return res
```

62. N皇后 (No.51) HARD

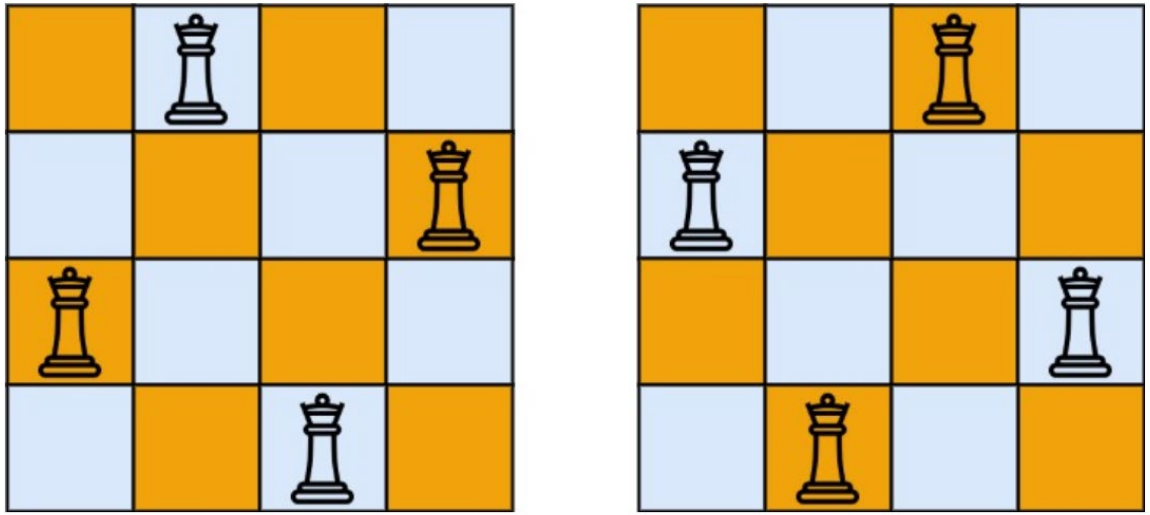
按照国际象棋的规则，皇后可以攻击与之处在同一行或同一列或同一斜线上的棋子。

n 皇后问题 研究的是如何将 `n` 个皇后放置在 `n×n` 的棋盘上，并且使皇后彼此之间不能相互攻击。

给你一个整数 `n` ，返回所有不同的 **n 皇后问题** 的解决方案。

每一种解法包含一个不同的 **n 皇后问题** 的棋子放置方案，该方案中 `'Q'` 和 `'.'` 分别代表了皇后和空位。

示例 1：



输入： n = 4
输出： [[".Q..","...Q","Q...","..Q."],["..Q.","Q...","...Q",".Q.."]]
解释： 如上图所示，4 皇后问题存在两个不同的解法。

示例 2：

输入： n = 1
输出： [["Q"]]

提示：

- `1 <= n <= 9`

62. N皇后 (No.51) HARD

思路

如果一个皇后放在(x,y)，那么所有满足(i=x)or(j=y)or(i+j=x+y)or(i-j=x-y)的全部都是互斥点。皇后选择一行行放进棋盘的话本身row是互斥的，因此用col,diag1(保存i+j),diag2(保存i-j)三个互斥标记的集合就能立刻排除所有非法点。

回溯的部分直接用模板即可，边界就是到了最后一个皇后，修改需要把棋盘对应位置改成皇后，再进行所有互斥标记，递归后复原互斥标记以及棋盘。

时间复杂度：$\mathcal{O}(n!)$

空间复杂度：$\mathcal{O}(n)$

```python
class Solution:
    def solveNQueens(self, n: int) -> List[List[str]]:
        def backtrack(pos):
            # 当前分支棋盘ans，第pos个皇后
            if pos == n:
                res.append(["".join(x) for x in board])
                return
            # 枚举所有可放皇后的位置
            for j in range(n):
                if j in col or pos+j in diag1 or pos-j in diag2:
                    continue
                board[pos][j] = 'Q'
                col.add(j), diag1.add(pos+j), diag2.add(pos-j)  # 互斥标记
                backtrack(pos+1)
                col.remove(j), diag1.remove(pos+j), diag2.remove(pos-j)  # 复原互斥标记
                board[pos][j] = '.'  # 棋盘复原

        res = []
        board = [['.' for _ in range(n)] for _ in range(n)]
        # diag1放下标i+j，diag2放下标i-j
        col, diag1, diag2 = set(), set(), set()
        backtrack(0)
        return res
```